

GeoLog

Plataforma de telemetria logística com persistência poliglota

Desafio Integrador · Implementação e Gerenciamento de Bancos de Dados NoSQL · UNIPÊ 2026.2

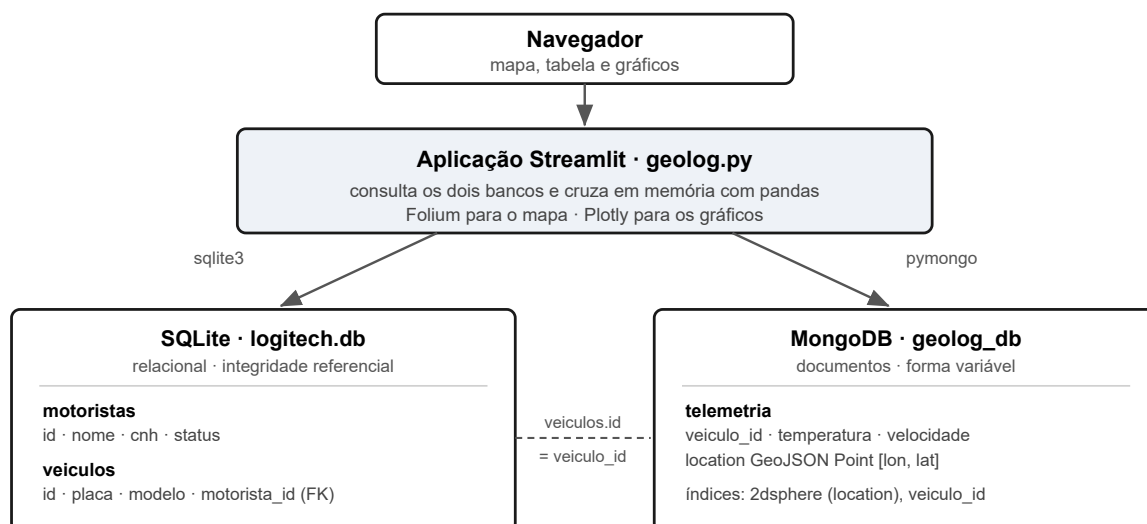
Prof. Me. Ricardo Roberto de Lima · Pedro Oliveira

1. O problema

A transportadora LogiTech Express tem dois tipos de dado com exigências opostas. O cadastro de motoristas e veículos precisa de integridade referencial e consulta relacional. A telemetria da frota chega em alta frequência, tem forma variável e precisa de consulta por proximidade geográfica. Um banco só atenderia mal um dos dois lados.

A solução é persistência poliglota: cada dado no banco que o resolve melhor, e a aplicação costurando os dois na memória. O cadastro fica em SQLite. A telemetria fica em MongoDB, com índice geoespacial `2dsphere`. A camada Streamlit lê dos dois e apresenta uma visão única.

2. Arquitetura



Arquitetura poliglota. A ligação entre os dois bancos não existe no banco: ela é feita pela aplicação, em memória, usando o identificador do veículo como chave.

Nenhum dos dois bancos conhece o outro. Não há chave estrangeira entre SQLite e MongoDB, e nenhum mecanismo garante que todo `veiculo_id` da telemetria exista no cadastro. Essa verificação é responsabilidade do código, e é o preço da arquitetura poliglota.

3. Os quatro módulos

Módulo 1 · Persistência e carga inicial

Na primeira execução a aplicação cria o esquema do SQLite, popula motoristas e veículos, e grava a telemetria no MongoDB. O índice `2dsphere` é criado por código na inicialização, como o enunciado exige, junto com um índice em `veiculo_id` para a leitura por veículo. A telemetria é gerada como histórico: o ponto do enunciado é tratado como a posição mais recente de cada veículo, e para trás são gerados pontos anteriores. Sem isso o gráfico de variação de temperatura do Módulo 4 teria um ponto só por veículo e não mostraria variação nenhuma.

Módulo 2 · Geoprocessamento e busca por raio

A busca acontece em dois passos. Primeiro uma agregação devolve a posição mais recente de cada veículo, porque a coleção guarda histórico e uma busca direta traria o mesmo veículo várias vezes, em posições antigas. Depois a distância é medida contra essas posições atuais. O operador `$near` opera sobre o índice `2dsphere` e `$maxDistance` é informado em metros, não em quilômetros. O resultado vai para um mapa Folium com o círculo do raio e um marcador por veículo, colorido por situação.

Módulo 3 · Visão unificada

O cruzamento é feito na aplicação. O SQLite devolve motorista e placa, o MongoDB devolve a última leitura de cada veículo, e os dois conjuntos são unidos em um `DataFrame` pela chave `veiculos.id = telemetria.veiculo_id`. É o ponto em que a arquitetura poliglota aparece de forma mais clara: o que em um banco único seria uma junção vira código.

Módulo 4 · Painel analítico

Três indicadores e dois gráficos. Os indicadores são veículos com motorista ativo, temperatura média da carga e contagem de alertas de velocidade acima de 80 km/h. Os gráficos mostram a variação de temperatura por veículo ao longo do tempo e a distribuição de status dos motoristas.

4. Consultas que sustentam a solução

Operação	Como é feita
Posição atual de cada veículo	Agregação com <code>\$sort</code> por data e <code>\$group</code> por <code>veiculo_id</code> , tomando o primeiro documento
Veículos dentro do raio	<code>\$near</code> com <code>\$geometry Point</code> e <code>\$maxDistance</code> em metros
Cadastro e frota	<code>SELECT</code> com <code>JOIN</code> entre <code>motoristas</code> e <code>veiculos</code> no SQLite
Visão unificada	<code>merge</code> dos dois resultados em memória, pela chave do veículo
Histórico de temperatura	Leitura ordenada por data, com projeção dos campos usados no gráfico

5. Decisões de projeto

A ordem das coordenadas

GeoJSON é sempre `[longitude, latitude]`, o contrário da forma como se fala. Invertido, o ponto vai parar em outro hemisfério e nenhuma consulta acusa erro, porque a coordenada continua válida. O código mantém essa ordem em um ponto só, na montagem do documento, para que o engano não possa se espalhar.

Por que a posição atual é agregação e não leitura direta

Como a coleção guarda histórico, um `find` simples traz várias leituras do mesmo veículo. Filtrar isso depois, na aplicação, funcionaria com três veículos e quebraria com trezentos. A agregação resolve no banco, que é onde o índice está.

A média de temperatura precisa de contexto

A média da carga entre veículos refrigerados e não refrigerados não descreve nenhum veículo real. O indicador foi mantido porque o enunciado pede, mas o gráfico de variação por veículo é o que permite uma leitura honesta: é nele que se vê um veículo específico saindo da faixa.

Reexecução do Streamlit

O Streamlit roda o script inteiro a cada interação. Conexão de banco é criada uma vez por processo e as consultas são guardadas em cache com tempo de validade, para que mover o raio no mapa não refaça toda a leitura nem abra conexão nova a cada clique.

6. Execução

Arquivo único `geolog.py`, como o enunciado exige. Dependências em `requirements.txt`.

Item	Versão verificada
Python	3.14
MongoDB	7, em contêiner
Bibliotecas	streamlit 1.62 · pymongo 4.17 · folium 0.20 · streamlit-folium 0.27 · plotly 7.0 · pandas 3.0

Com o MongoDB acessível, a aplicação sobe com `streamlit run geolog.py` e semeia os dois bancos sozinha na primeira execução. Testada de ponta a ponta, sem exceção, erro ou aviso.

7. Limites conhecidos

- Não há verificação de que todo `veiculo_id` da telemetria exista no cadastro. Um identificador órfão aparece no mapa e some da visão unificada, sem aviso.
- O cruzamento em memória atende o volume do estudo de caso. Em escala real, a visão unificada passaria a ser materializada, e não recalculada a cada leitura.
- A geração de telemetria é sintética e serve para demonstrar as consultas, não para análise.